

A I 应用部署、评测与落地

从训练好的模型到生产级服务，A I 工程化落地需要解决推理优化、服务部署、评测监控三大挑战。这是 A I 项
" 最后一公里 " 的关键——一个训练精度 9 9 % 的模型，如果部署不当导致延迟 3 秒，在实际业务中可能完全不
。

A I 应用部署、评测与落地

从训练好的模型到生产级服务，A I 工程化落地需要解决推理优化、服务部署、评测监控三大挑战。这是 A I 项
" 最后一公里 " 的关键——一个训练精度 9 9 % 的模型，如果部署不当导致延迟 3 秒，在实际业务中可能完全不
。

- - -

一、模型部署的挑战

1 . 1 三大核心挑战

挑战 说明 典型场景 解决方向

硬件异构 C P U / G P U / N P U / 手机芯片，不同硬件需不同优化 同一模型需适配服务器和手机
统一中间格式 + 硬件适配层

性能瓶颈 推理延迟、吞吐量、显存占用 实时对话延迟 < 2 0 0 m s 量化、缓存、批处理

跨平台兼容 P y T o r c h 模型无法直接部署到嵌入式 需要转换为通用格式 O N N X / T F L i t e 等中

服务可靠性 模型推理可能失败、超时、结果异常 线上服务 9 9 . 9 % 可用性 重试、降级、熔断

版本管理 模型频繁迭代，需要灰度发布和回滚 每周上线新模型版本 模型注册中心 + A / B 测试

成本控制 G P U 资源昂贵，需要高效利用 单张 A 1 0 0 约 2 万美元 量化、批处理、弹性伸缩

1 . 2 部署方案全景图

方案 定位 优势 适用场景

O N N X R u n t i m e 跨框架推理引擎 兼容性广、C P U 优化好 通用跨平台部署

T e n s o r R T N V I D I A G P U 专用加速 极致性能（2 - 5 x 加速） G P U 生产环境

O p e n V I N O I n t e l 硬件优化 C P U 友好、边缘设备 I n t e l 服务器 / I o T

T F L i t e 移动端推理 安卓 / i O S 支持 手机 A P P

N C N N / M N N 国产移动端框架 极致轻量 移动端 / 嵌入式

v L L M / T G I L L M 专用推理 高吞吐、P a g e d A t t e n t i o n 大语言模型服务

1 . 3 部署架构演进

第一代：单机单卡部署

训练模型 → 直接加载推理 → 单点服务

问题：无弹性、无容灾、性能差

第二代：容器化 + 负载均衡

Docker 封装 → K8s 编排 → 多副本负载均衡

改进：弹性伸缩、滚动更新、资源隔离

第三代：微服务 + 推理优化

模型优化（量化 / 蒸馏） → 推理引擎（TRT / ONNX） → 服务网格 → 可观测性

改进：极致性能、精细监控、灰度发布

第四代：MLOps 全流程

模型注册 → 自动优化 → 灰度部署 → 监控告警 → 自动回滚

改进：全链路自动化、数据驱动迭代

- - -

二、ONNX 详解

2.1 模型导出

```
import torch
import torchvision
```

加载模型

```
model = torchvision.models.resnet50(pretrained=True)
model.eval()
```

创建示例输入

```
dummy_input = torch.randn(1, 3, 224, 224)
```

导出ONNX

```
torch.onnx.export(
    model,
    dummy_input,
```

```

"resnet50.onnx",
inputnames=["input"],
outputnames=["output"],
dynamicaxes={
    "input": {0: "batchsize"}, # 动态batch
    "output": {0: "batchsize"}
},
opsetversion=14, # 使用14+版本
doconstantfolding=True, # 常量折叠优化
)

```

验证ONNX模型

```

import onnx
onnxmodel = onnx.load("resnet50.onnx")
onnx.checker.checkmodel(onnxmodel)
print(f"模型输入: {[inp.name for inp in onnxmodel.graph.input]}")
print(f"模型输出: {[out.name for out in onnxmodel.graph.output]}")

```

2.2 常见导出问题

问题 原因 解决方案

不支持的算子 PyTorch算子ONNX没有 自定义算子注册或替代实现

动态形状 某些操作不支持动态shape 指定固定shape或使用dynamicaxes

精度不一致 浮点运算顺序差异 验证数值差异在可接受范围

大模型导出OOM 导出过程需要额外显存 使用CPU导出

2.3 ONNX Runtime 推理

```

import onnxruntime as ort
import numpy as np

```

配置选项

```

sessoptions = ort.SessionOptions()
sessoptions.graphoptimizationlevel = ort.GraphOptimizationLevel.ORT_OPTIMIZATION_LEVEL_ALL
sessoptions.intraopnumthreads = 4 # CPU线程数

```

创建推理会话

```
session = ort.InferenceSession(
    "model.onnx",
    sessoptions,
    providers=["CUDAExecutionProvider", "CPUExecutionProvider"]
)
```

获取输入输出信息

```
inputinfo = session.getinputs()[0]
outputinfo = session.getoutputs()[0]
print(f"输入: {inputinfo.name}, 形状: {inputinfo.shape}")
print(f"输出: {outputinfo.name}, 形状: {outputinfo.shape}")
```

推理

```
inputdata = np.random.randn(1, 3, 224, 224).astype(np.float32)
results = session.run(
    [outputinfo.name],
    {inputinfo.name: inputdata}
)
print(f"输出形状: {results[0].shape}")
```

2.4 ONNX 量化

```
from onnxruntime.quantization import quantizedynamic
```

动态量化（简单快速）

```
quantizedynamic(
    modelinput="model.onnx",
    modeloutput="modelquantized.onnx",
    weighttype=QuantType.QUInt8 # 权重量化为8位无符号整数
)
```

静态量化（精度更高，需要校准数据）

```
from onnxruntime.quantization import CalibrationData
```

```

class MyCalibrationReader(CalibrationDataReader):
    def __init__(self, calibrationdata):
        self.data = calibrationdata
        self.idx = 0

    def getnext(self):
        if self.idx >= len(self.data):
            return None
        data = {inputname: self.data[self.idx]}
        self.idx += 1
        return data

quantizestatic(
    modelinput="model.onnx",
    modeloutput="modelstaticquant.onnx",
    calibrationdatareader=MyCalibrationReader(calibdata),
    quantformat=QuantType.QInt8,
)

- - -

```

三、TensorRT 详解

3.1 为什么TensorRT能大幅加速？

TensorRT的核心优化原理：

优化技术 原理 典型加速效果

层融合 (Layer Fusion) 将Conv+BN+ReLU融合为一个kernel，减少显存访问

精度校准 (Precision Calibration) FP32→FP16 / INT8，减少计算量和显存

核选择 (Kernel Auto-Tuning) 针对当前GPU选择最优的CUDA核实现 1.1 - 1.2x

动态显存管理 复用中间张量的显存，避免重复分配 减少显存30 - 50%

多流执行 无依赖的算子并行执行 1.1 - 1.2x

3.2 完整工作流程

PyTorch模型 → ONNX → TensorRT Engine → 推理

步骤：

1. 导出ONNX模型
2. 解析ONNX构建TensorRT网络
3. 优化层融合、核选择
4. 生成序列化的Engine
5. 反序列化Engine进行推理

3.2 Python API 使用

```
import tensorrt as trt
```

创建Builder

```
logger = trt.Logger(trt.Logger.WARNING)
builder = trt.Builder(logger)
```

创建网络

```
network = builder.create_network(1 << int(trt.NetworkDefinitionCreationFlag_MAX_BATCH_SIZE))
```

解析ONNX模型

```
parser = trt.OnnxParser(network, logger)
with open("model.onnx", "rb") as f:
    success = parser.parse(f.read())
if not success:
    for i in range(parser.num_errors):
        print(f"解析错误: {parser.get_error(i)}")
```

配置构建器

```
config = builder.create_builder_config()
config.set_memory_pool_limit(trt.MemoryPoolType.WORKING, 1024 * 1024 * 1024)
```

FP16精度

```
config.set_flag(trt.BuilderFlag.FP16)
```

INT8量化（需要校准器）

```
config.set_flag(trt.BuilderFlag.INT8)
```

```
config.int8calibrator = MyCalibrator()
```

动态形状配置

```
profile = builder.createoptimizationprofile()  
profile.setshape("input",  
min=(1, 3, 224, 224), # 最小shape  
opt=(8, 3, 224, 224), # 最优shape  
max=(32, 3, 224, 224)) # 最大shape  
config.addoptimizationprofile(profile)
```

构建引擎

```
engine = builder.buildserializednetwork(network,
```

保存引擎

```
with open("model.trt", "wb") as f:  
f.write(engine)
```

3.3 TensorRT 推理

```
import tensorrt as trt  
import pycuda.driver as cuda  
import pycuda.autoinit
```

```
class TensorRTInference:
```

```
def init(self, enginepath):
```

```
# 加载引擎
```

```
runtime = trt.Runtime(trt.Logger(trt.Logger.WARNING))
```

```
with open(enginepath, "rb") as f:
```

```
self.engine = runtime.deserializecudaengine(f.read())
```

```
self.context = self.engine.createexecutioncontext()
```

```
# 分配显存
```

```
self.inputs = []
```

```
self.outputs = []
```

```
self.bindings = []
```

```
self.stream = cuda.Stream()
```

```

for i in range(self.engine.numiotensors):
    name = self.engine.gettensorname(i)
    dtype = trt.nptype(self.engine.gettensordtype(name))
    shape = self.engine.gettensorshape(name)
    size = trt.volume(shape)

    # 分配GPU内存
    hostmem = cuda.pagelockedempty(size, dtype)
    devicemem = cuda.memalloc(hostmem.nbytes)

    self.bindings.append(int(devicemem))

    if self.engine.gettensormode(name) == trt.TensorModeIO:
        self.inputs.append({'host': hostmem, 'device': devicemem})
    else:
        self.outputs.append({'host': hostmem, 'device': devicemem})

def infer(self, inputdata):
    # 复制输入到GPU
    np.copyto(self.inputs[0]['host'], inputdata.ravel())
    for inp in self.inputs:
        cuda.memcpy_htod_async(inp['device'], inp['host'], self.stream)

    # 执行推理
    self.context.execute_async_v3(streamhandle=self.stream)

    # 复制输出到CPU
    for out in self.outputs:
        cuda.memcpy_dtoh_async(out['host'], out['device'], self.stream)
    self.stream.synchronize()

    return self.outputs[0]['host'].reshape(self.outputs[0].shape)

```

3.4 性能优化策略

策略 原理 效果

层融合 自动融合 Conv + ReLU、Conv + BN + ReLU 减少显存访问和 kernel 启动
FP16 半精度推理 ~ 2 倍加速, 精度损失极小
INT8 8 位整型推理 ~ 4 倍加速, 需要校准
动态 batch 支持不同 batch size 推理 灵活适配
流式处理 多流并行推理 提高 GPU 利用率

3.5 trtexec 命令行工具

trtexec 是 TensorRT 自带的基准测试工具, 无需编写代码即可快速评估模型性能:

ONNX → TensorRT Engine (FP16)

```
trtexec --onnx=model.onnx --fp16 --saveEngine=model
```

测量推理性能

```
trtexec --loadEngine=model.fp16.trt --batch=1 --it
```

查看详细性能信息

```
trtexec --onnx=model.onnx --fp16 --verbose
```

动态 batch 推理

```
trtexec --onnx=model.onnx --fp16 \
--minShapes=input:1x3x224x224 \
--optShapes=input:8x3x224x224 \
--maxShapes=input:32x3x224x224
```

输出解读:

Throughput: 1234.56 qps ← 吞吐量 (每秒处理请求数)
Latency: mean=0.81ms P99=1.23ms ← 延迟
GPU Compute Time: 0.65ms ← 纯 GPU 计算时间
H2D / D2H Transfer: 0.10ms ← 数据传输时间

3.6 TensorRT 常见问题排查

问题 原因 解决方案

构建引擎非常慢 网络复杂, 核选择耗时长 使用 --tacticSources 限制搜索范围

FP16精度不够 某些层对精度敏感 对敏感层强制FP32：`config.setflag(trt.E`
 不支持某算子 TensorRT版本不支持 升级TRT版本或实现自定义插件
 显存不足 模型+KV Cache太大 使用量化或减小`max_batch_size`
 动态shape性能差 每次shape变化需重新优化 预编译多个profile
 - - -

四、模型量化技术

4.1 量化原理

均匀量化：将FP32值映射到INT8值

$$x_{int} = \text{round}\left(\frac{x_{fp}}{scale}\right)$$

$$x_{fp} = (x_{int} - zero_point) \times scale$$

4.2 量化类型对比

类型 精度 速度 显存 精度损失 复杂度

FP32	32位	1×	基准	无	—
FP16	16位	~2×	-50%	极小	低
BF16	16位	~2×	-50%	极小	低
INT8	8位	~4×	-75%	小	中
INT4	4位	~8×	-87%	中等	高

4.3 LLM 量化方案

方案 原理 精度保持 推理速度 适用场景

GPTQ	基于Hessian信息的最优量化	较好	~3.5×	GPU推理
AWQ	保护重要权重通道	好	~3.5×	GPU推理
GGUF / llama.cpp	CPU友好的量化格式	好	依赖硬件	CPU / 边缘推理
SmoothQuant	平滑激活值再量化	好	~2×	GPU推理

4.4 GPTQ 量化实战

```
from autogptq import AutoGPTQForCausalLM, BaseQuantizer
from transformers import AutoTokenizer
```

量化配置

```
quantizeconfig = BaseQuantizeConfig(
    bits=4, # 4-bit量化
    groupsize=128, # 分组大小
    descact=True, # 激活排序（提升精度但更慢）
)
```

加载模型

```
model = AutoGPTQForCausalLM.from_pretrained(
    "Qwen/Qwen2.5-7B",
    quantizeconfig=quantizeconfig
)
tokenizer = AutoTokenizer.from_pretrained("Qwen/Qwen2.5-7B")
```

准备校准数据

```
calibdata = loadcalibrationdata(nsamples=128)
```

量化

```
model.quantize(calibdata)
```

保存

```
model.savequantized("./qwen-7b-gptq-4bit")
```

4.5 AWQ 量化实战

AWQ (Activation-aware Weight Quantization) 的核心创新：不是所有权重同等重要，保护 1% 的 "显著" 权重通道就能保持量化精度。

```
from awq import AutoAWQForCausalLM
from transformers import AutoTokenizer
```

加载模型

```
model = AutoAWQForCausalLM.from_pretrained("Qwen/Qwen2.5-7B")
tokenizer = AutoTokenizer.from_pretrained("Qwen/Qwen2.5-7B")
```

量化配置

```
quantconfig = {  
    "zeropoint": True, # 使用非对称量化  
    "qgroupsize": 128, # 分组大小  
    "wbit": 4, # 权重位宽  
    "version": "GEMM", # 使用GEMM核（更快）  
}
```

量化

```
model.quantize(tokenizer, quantconfig=quantconfig)
```

保存

```
model.savequantized("./qwen-7b-awq-4bit")
```

4.6 GGUF / llama.cpp 量化——CPU和边缘设备的最佳选择

GGUF 是 llama.cpp 使用的量化格式，专为CPU推理优化，支持多种量化精度：

GGUF 量化级别（按精度从高到低）：

量化类型	位宽	模型大小 (7B)	精度损失	适用场景
------	----	-----------	------	------

Q80	8bit	~7.7GB	极小	内存充足
-----	------	--------	----	------

Q5KM	5bit	~4.8GB	小	平衡选择
------	------	--------	---	------

Q4KM	4bit	~4.1GB	中等	推荐默认
------	------	--------	----	------

Q3KM	3bit	~3.5GB	较大	内存紧张
------	------	--------	----	------

Q2K	2bit	~2.9GB	大	极端资源受限
-----	------	--------	---	--------

安装 llama.cpp

```
git clone https://github.com/ggerganov/llama.cpp  
cd llama.cpp && make
```

转换 HuggingFace 模型为 GGUF

```
python convert_hf_to_gguf.py /path/to/model --outfile
```

量化为 Q4KM

```
./llama-quantize model.gguf model-Q4KM.gguf Q4KM
```

推理

```
. / llama-cli -m model-Q4KM.gguf -p "解释什么是机器学习" -n
```

启动OpenAI兼容API服务

```
. / llama-server -m model-Q4KM.gguf --host 0.0.0.0
```

4.7 量化方案选择指南

使用场景 推荐方案 理由

GPU服务器、追求极致速度 GPTQ / AWQ 4bit GPU加速好，吞吐高

CPU服务器 / 个人电脑 GGUF Q4KM llama.cpp CPU优化好

边缘设备 / 手机 GGUF Q2K / Q3 极致压缩

需要精度保证 AWQ > GPTQ 保护重要通道

快速验证 / 原型 llama.cpp直接量化 无需GPU，一键完成

4.8 模型剪枝 (Pruning)

量化减少每个参数的位宽，剪枝则直接删除不重要的参数：

剪枝类型：

1. 非结构化剪枝 (Unstructured Pruning)

- 将单个权重置零（不删除，只是0）
- 压缩需要稀疏矩阵格式支持
- 精度损失小，但加速效果依赖硬件稀疏计算支持

2. 结构化剪枝 (Structured Pruning)

- 删除整个神经元 / 通道 / 层
- 直接减少计算量，无需特殊硬件支持
- 精度损失较大，需要微调恢复

3. LLM剪枝 (如SparseGPT, Wanda)

- 针对大语言模型的专用剪枝方法
- SparseGPT：基于Hessian信息一次性剪枝50%权重
- Wanda：基于权重和激活幅值剪枝，无需重训练

PyTorch 非结构化剪枝示例

```
import torch.nn.utils.prune as prune
```

对线性层剪枝30%

```
module = model.fc
```

```
prune.l1unstructured(module, name="weight", amount=0.3)
```

查看剪枝后的权重

```
print(torch.sum(module.weight == 0)) # 约30%为零
```

永久化剪枝（移除原始权重，只保留稀疏权重）

```
prune.remove(module, "weight")
```

4.9 知识蒸馏 (Knowledge Distillation)

用一个大的 " 教师模型 " 指导小的 " 学生模型 " 训练，使小模型获得接近大模型的性能：

蒸馏核心原理：

教师模型 (大) 学生模型 (小)

```
logitst   logitss
```

└──────── K L 散度损失 ─────────┘

总损失 = $\alpha \times \text{蒸馏损失} + (1 - \alpha) \times \text{硬标签损失}$

蒸馏损失 = $\text{KLdiv}(\text{softmax}(\text{logitss} / T), \text{softmax}(\text{logitst}))$

- T 是温度参数 ($T > 1$ 使概率分布更平滑，暴露更多 " 暗知识 ")
- α 通常取 0.7 - 0.9

```
class DistillationTrainer:
```

```
    """ 知识蒸馏训练器 """
```

```
    def __init__(self, teacher, student, temperature=4.0,
```

```
                 self.teacher = teacher
```

```
                 self.student = student
```

```
                 self.T = temperature
```

```

self.alpha = alpha

# 教师模型冻结
for param in self.teacher.parameters():
    param.requiresgrad = False
self.teacher.eval()

def distillationloss(self, studentlogits, teacherlogits):
    """蒸馏损失 = KL散度 + 交叉熵"""
    # 软标签损失（蒸馏损失）
    softloss = nn.KLDivLoss(reduction='batchmean')(
        F.logsoftmax(studentlogits / self.T, dim=1),
        F.softmax(teacherlogits / self.T, dim=1)
    ) * (self.T ** 2) # 温度补偿

    # 硬标签损失
    hardloss = nn.CrossEntropyLoss()(studentlogits, labels)

    return self.alpha * softloss + (1 - self.alpha) * hardloss

def trainstep(self, inputs, labels):
    with torch.no_grad():
        teacherlogits = self.teacher(inputs)

    studentlogits = self.student(inputs)
    loss = self.distillationloss(studentlogits, teacherlogits)
    return loss

```

典型效果：

教师模型（7 B）： 准确率92%

学生模型（1.8 B，直接训练）： 准确率85%

学生模型（1.8 B，蒸馏）： 准确率89% ← 接近教师模型！

- - -

五、LLM 部署方案

5.1 vLLM (高吞吐LLM推理)

核心创新：Paged Attention

传统KV Cache需要预分配连续显存，导致碎片化。Paged Attention将KV Cache分为固定大小的“页”，按需分配，类似操作系统的虚拟内存。

传统KV Cache：

预分配最大长度显存 → 80%显存浪费（实际序列通常远短于最大长度）

显存碎片化 → 无法服务更多请求

Paged Attention：

按需分配KV Cache页 → 显存利用率接近100%

类似OS虚拟内存 → 支持跨请求共享公共前缀（如system prompt）

物理页表：[页0：(K0, V0)] [页1：(K1, V1)] [页2：(K2, V2)] ...

逻辑序列：seq0 → [页0, 页1]（短序列只需2页）

seq1 → [页3, 页4, 页5, 页6]（长序列需要4页）

```
from vllm import LLM, SamplingParams
```

启动vLLM

```
llm = LLM(  
    model="Qwen/Qwen2.5-7B",  
    tensor_parallel_size=1, # GPU数量  
    gpu_memory_utilization=0.9, # GPU显存利用率  
    max_model_len=4096, # 最大序列长度  
    quantization="awq", # 可选量化方式  
    enable_prefix_caching=True, # 启用前缀缓存（相同system prompt）  
    enable_chunked_prefill=True, # 分块预填充（降低首token延迟）  
)
```

批量推理

```
sampling_params = SamplingParams(  
    temperature=0.7,  
    top_p=0.9,  
    max_tokens=512,
```



```
frequencypenalty=0.0, # 频率惩罚
presencepenalty=0.0, # 存在惩罚
stop=["\n\n"], # 停止词
)
```

```
prompts = [
    "请解释什么是机器学习",
    "Python和Java的区别是什么",
    "深度学习的优势有哪些",
]
```

```
outputs = llm.generate(prompts, samplingparams)
for output in outputs:
    print(output.outputs[0].text)
```

vLLM OpenAI兼容API服务：

启动API服务（兼容OpenAI API格式）

```
python -m vllm.entrypoints.openai.api_server \
--model Qwen/Qwen2.5-7B \
--host 0.0.0.0 \
--port 8000 \
--tensor-parallel-size 1 \
--gpu-memory-utilization 0.9 \
--max-model-len 4096 \
--quantization awq
```

客户端调用（与OpenAI SDK完全兼容）

```
from openai import OpenAI
client = OpenAI(baseurl="http://localhost:8000/v1")

response = client.chat.completions.create(
    model="Qwen/Qwen2.5-7B",
    messages=[{"role": "user", "content": "解释什么是深度学习"}],
    temperature=0.7,
```

```
max_tokens=512,
)
print(response.choices[0].message.content)
```

流式输出

```
stream = client.chat.completions.create(
model="Qwen/Qwen2.5-7B",
messages=[{"role": "user", "content": "写一首诗"}],
stream=True,
)
for chunk in stream:
if chunk.choices[0].delta.content:
print(chunk.choices[0].delta.content, end=" ")
```

vLLM vs 其他方案：

特性 vLLM TGI Ollama

核心优势 高吞吐 生产级功能 易用性

PagedAttention

Continuous Batching

多GPU

量化支持 GPTQ/AWQ GPTQ/AWQ GGUF

OpenAI兼容API

Speculative Decoding

适用场景 高并发生产 企业部署 本地开发

5.2 Speculative Decoding (投机解码)

大模型推理的瓶颈在于自回归生成必须串行（生成第 i 个 token 依赖前 $i - 1$ 个 token）。Speculative Decoding 的核心思想：用小模型快速 "猜测" 多个 token，再用大模型并行验证。

传统自回归解码：

大模型逐个生成： $t_1 \rightarrow t_2 \rightarrow t_3 \rightarrow t_4 \rightarrow t_5$ （5次前向传播）

Speculative Decoding：

1. 小模型快速猜测： $t_1', t_2', t_3', t_4', t_5'$ （1次前向传播）

2. 大模型并行验证：验证 $t_1' - t_5'$ 是否正确（1次前向传播）
3. 保留正确前缀，从第一个错误处重新生成

如果小模型猜测5个token中有4个正确 → 2次前向传播生成5个token

加速比： $5 / 2 = 2.5x$ ！

关键：大模型验证时的概率分布必须和小模型的分布对齐，
否则小模型“猜对”的token也可能被拒绝。

```
vLLM 使用 Speculative Decoding
llm = LLM(
    model = "Qwen/Qwen2.5-7B", # 大模型（验证者）
    speculativemodel = "Qwen/Qwen2.5-0.5B", # 小模型（猜测者）
    numspeculativetokens = 5, # 每次猜测5个token
    speculativemaxmodel len = 4096,
)
```

典型加速效果： $1.5x - 3x$ （取决于任务和模型匹配度）

5.3 Ollama（本地部署）

安装

```
curl -fsSL https://ollama.com/install.sh sh
```

下载模型

```
ollama pull qwen2.5:7b
```

运行

```
ollama run qwen2.5:7b
```

API调用

```
curl http://localhost:11434/api/generate -d '{
    "model": "qwen2.5:7b",
    "prompt": "什么是深度学习？"
}'
```

OpenAI兼容API

```
curl http://localhost:11434/v1/chat/completions -
```

```
"model": "qwen2.5:7b",
"messages": [{ "role": "user", "content": "什么是深度学习"
}]'
```

自定义模型导入：

将GGUF模型导入Ollama

1. 创建Model file

```
cat > Model file <<EOF
FROM ./model-Q4KM.gguf
PARAMETER temperature 0.7
PARAMETER topp 0.9
PARAMETER numctx 4096
SYSTEM " " "你是一个专业的AI助手，请用中文回答问题。" " "
EOF
```

2. 创建模型

```
ollama create my-model -f Model file
```

3. 运行

```
ollama run my-model
```

5.4 LLM 推理性能关键指标

指标 全称 含义 典型要求

TTFT Time To First Token 从请求到第一个token的延迟 < 500ms (双

TPS Tokens Per Second 每秒生成的token数(单请求) > 30 TPS

Throughput 吞吐量 每秒处理的请求 / token数 视业务定

Latency P99 99分位延迟 99%请求的响应时间 < 2s

Concurrent 并发数 同时服务的请求数 越高越好

推理过程分解：

用户请求 → [Prefill阶段] → [Decode阶段] → 返回结果

处理全部输入prompt 逐个生成token

计算密集、可并行 显存带宽瓶颈

T T F T 主要由此决定 T P S 主要由此决定

优化思路：

- 降低T T F T：分块预填充 (C h u n k e d P r e f i l l)、前缀缓存
- 提高T P S：量化减少显存带宽需求、K V C a c h e 压缩
- 提高吞吐：C o n t i n u o u s B a t c h i n g、动态批处理
- - -

六、服务化部署

6.1 F a s t A P I 推理服务

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import onnxruntime as ort
import numpy as np
from typing import List
import asyncio
from concurrent.futures import ThreadPoolExecutor

app = FastAPI(title="AI Inference Service", version="1.0.0")
```

全局加载模型（启动时加载）

```
session = ort.InferenceSession("model.onnx", providers=["CPU"])
```

线程池（避免推理阻塞事件循环）

```
executor = ThreadPoolExecutor(max_workers=4)
```

```
class PredictRequest(BaseModel):
```

```
    inputdata: List[List[float]]
```

```
class PredictResponse(BaseModel):
```

```
    prediction: List[List[float]]
```

```
    confidence: float
```

```
    latencyms: float
```

```

@app.post("/predict", response_model=PredictResponse)
async def predict(request: PredictRequest):
    import time
    start = time.perfcounter()

    try:
        inputarray = np.array(request.inputdata, dtype=np.float32)
        inputname = session.getinputs()[0].name

        # 在线程池中执行推理（避免阻塞）
        loop = asyncio.geteventloop()
        outputs = await loop.run_in_executor(
            executor,
            lambda: session.run(None, {inputname: inputarray})
        )

        prediction = outputs[0].tolist()
        confidence = float(np.max(outputs[0]))
        latency = (time.perfcounter() - start) * 1000

        return PredictResponse(
            prediction=prediction,
            confidence=confidence,
            latencyms=round(latency, 2)
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.get("/health")
async def healthcheck():
    return {"status": "healthy", "modelloaded": True}

启动: uvicorn api:app --host 0.0.0.0 --port 8000 --

```

6.2 Triton Inference Server (NVIDIA推理服务器)

T r i t o n 是 N V I D I A 开源的生产级推理服务器，支持多框架、多模型并发、动态批处理：

T r i t o n 核心特性：

- 1 . 多框架支持：T e n s o r R T、O N N X、P y T o r c h、T e n s o r F l o w
- 2 . 动态批处理：自动将多个请求合并为一个 b a t c h，提高吞吐
- 3 . 多模型并发：同一 G P U 上并行运行多个模型
- 4 . 模型版本管理：支持多版本模型同时服务
- 5 . 健康检查与监控：内置 m e t r i c s 端点
- 6 . 请求调度：优先级队列、超时处理

T r i t o n 模型仓库结构

```
model repository /
├── resnet50 /
│   ├── config.pbtxt # 模型配置
│   └── 1 / # 版本1
│       ├── model.onnx # 或 model.plan (TensorRT)
├── bertner /
│   ├── config.pbtxt
│   └── 1 /
│       └── model.onnx
```

config.pbtxt 示例

```
name: "resnet50"
platform: "onnxruntimeonnx"
maxbatchsize: 32

input [
{
name: "input"
datatype: TYPE_FP32
dims: [3, 224, 224]
}
]
```

```
output [
{
name: "output"
datatype: TYPEFP32
dims: [1000]
}
]
```

动态批处理配置

```
dynamicbatching {
maxqueuedelaymicroseconds: 10000 # 最多等10ms凑batch
preferredbatchsize: [8, 16, 32] # 优先的batch大小
}
```

实例组（GPU / CPU分配）

```
instancegroup [
{
count: 2 # 2个推理实例
kind: KINDGPU # 使用GPU
gpus: [0] # GPU 0
}
]
```

启动Triton

```
docker run --gpus all --rm -p 8000:8000 -p 8001:8001 \
-v $(pwd)/modelrepository:/models \
nvcrl.io/nvidia/tritonserver:24.01-py3 \
tritonserver --model-repository=/models
```

HTTP推理请求

```
curl localhost:8000/v2/models/resnet50/infer \
-d '{"inputs": [{"name": "input", "shape": [1, 3, 224, 224]}]}'
```

查看模型状态


```
curl localhost:8000/v2/models/resnet50
```

查看性能指标

```
curl localhost:8002/metrics
```

6.3 LLM 推理服务完整实现

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import List, Optional
import time
import asyncio

app = FastAPI(title="LLM Inference Service")

===== 请求 / 响应模型 =====
class ChatMessage(BaseModel):
    role: str
    content: str

class ChatRequest(BaseModel):
    messages: List[ChatMessage]
    temperature: float = 0.7
    max_tokens: int = 512
    stream: bool = False

class ChatResponse(BaseModel):
    content: str
    model: str
    usage: dict
    latencyms: float

===== 模型管理器 =====
class ModelManager:
    """ 管理模型的加载、推理和卸载 """

    def __init__(self):
```

```

self.model = None
self.tokenizer = None
self.modelname = ""
self.lock = asyncio.Lock()

async def loadmodel(self, modelpath: str):
    """加载模型（异步，避免阻塞）"""
    async with self.lock:
        loop = asyncio.geteventloop()
        self.model, self.tokenizer = await loop.run_in_executor(
            None, self.loadsync, modelpath
        )
        self.modelname = modelpath
        print(f"模型加载完成: {modelpath}")

    def loadsync(self, modelpath):
        from transformers import AutoModelForCausalLM, AutoTokenizer
        import torch
        tokenizer = AutoTokenizer.from_pretrained(modelpath)
        model = AutoModelForCausalLM.from_pretrained(
            modelpath,
            torch_dtype=torch.bfloat16,
            device_map="auto"
        )
        model.eval()
        return model, tokenizer

    async def generate(self, messages, *kwargs):
        """生成推理"""
        start = time.perfcounter()

        # 构造prompt
        text = self.tokenizer.apply_chat_template(
            messages, tokenize=False, add_generation_prompt=True
        )

```

```

inputs = self.tokenizer(text, return_tensors="pt")

# 异步推理
loop = asyncio.geteventloop()
outputs = await loop.run_in_executor(
None, self.generate_sync, inputs, kwargs
)

latency = (time.perfcounter() - start) * 1000

result = self.tokenizer.decode(outputs[0][inputs.
skip_special_tokens=True)

return {
"content": result,
"latency_ms": round(latency, 2),
"prompt_tokens": inputs.input_ids.shape[1],
"completion_tokens": outputs.shape[1] - inputs.inp
}

def generate_sync(self, inputs, kwargs):
import torch
with torch.no_grad():
return self.model.generate(
*inputs,
max_new_tokens=kwargs.get('max_tokens', 512),
temperature=kwargs.get('temperature', 0.7),
do_sample=True,
topp=0.9,
)

```

全局模型管理器

```

manager = ModelManager()

@app.onevent("startup")
async def startup():

```

```

await manager.loadmodel("Qwen/Qwen2.5-7B")

@app.post("/v1/chat/completions", response_model=ChatCompletion)
async def chatcompletions(request: ChatRequest):
    try:
        messages = [{"role": m.role, "content": m.content} for m in request.messages]
        result = await manager.generate(
            messages,
            temperature=request.temperature,
            maxtokens=request.maxtokens
        )
        return ChatResponse(
            content=result["content"],
            model=manager.modelname,
            usage={
                "prompttokens": result["prompttokens"],
                "completiontokens": result["completiontokens"],
            },
            latencyms=result["latencyms"]
        )
    except Exception as e:
        raise HTTPException(statuscode=500, detail=str(e))

```

6.4 Docker 容器化

===== CV模型推理镜像 =====

```
FROM python:3.10-slim
```

安装系统依赖

```
RUN apt-get update && apt-get install -y libgl1-mesa-dev \
    /var/lib/apt/lists/
```

安装Python依赖

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

复制模型和代码

```
COPY model.onnx /app/  
COPY api.py /app/
```

```
WORKDIR /app
```

健康检查

```
HEALTHCHECK --interval=30s --timeout=5s \\  
CMD curl -f http://localhost:8000/health exit 1
```

启动命令

```
CMD ["uvicorn", "api:app", "--host", "0.0.0.0", "
```

===== LLM推理镜像 (GPU) =====

```
FROM nvcr.io/nvidia/pytorch:24.01-py3
```

安装vLLM

```
RUN pip install vllm
```

复制模型和配置

```
COPY modelconfig.yaml /app/
```

```
WORKDIR /app
```

启动vLLM API服务

```
CMD ["python", "-m", "vllm.entrypoints.openai.api\  
"--model", "/models/qwen-7b", \  
"--host", "0.0.0.0", "--port", "8000", \  
"--tensor-parallel-size", "1"]
```

docker-compose.yml —— 多模型服务编排

```
version: '3.8'
```

```
services:
```

```
cv-service:
```

```
build: ./cvservice
```

```
ports:
```

- "8001:8000"

deploy:

resources:

reservations:

devices:

- driver: nvidia

count: 1

capabilities: [gpu]

restart: unless-stopped

llm-service:

build: ./llmservice

ports:

- "8002:8000"

volumes:

- ./models:/models

deploy:

resources:

reservations:

devices:

- driver: nvidia

count: 1

capabilities: [gpu]

restart: unless-stopped

nginx:

image: nginx:alpine

ports:

- "80:80"

volumes:

- ./nginx.conf:/etc/nginx/nginx.conf

dependson:

- cv-service

- llm-service

6.5 Kubernetes 部署

```
k8s-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: llm-inference
  labels:
    app: llm-inference
spec:
  replicas: 2 # 2个副本
  selector:
    matchLabels:
      app: llm-inference
  template:
    metadata:
      labels:
        app: llm-inference
    spec:
      containers:
        - name: llm-server
          image: registry.example.com/llm-server:latest
          ports:
            - containerPort: 8000
          resources:
            limits:
              nvidia.com/gpu: 1 # 每个Pod 1个GPU
            memory: "16Gi"
          requests:
            nvidia.com/gpu: 1
            memory: "8Gi"
          env:
            - name: MODEL_PATH
```

```
value: "/models/qwen-7b"
- name: MAXMODELLEN
value: "4096"
volumeMounts:
- name: model-volume
mountPath: /models
livenessProbe: # 存活探针
httpGet:
path: /health
port: 8000
initialDelaySeconds: 60
periodSeconds: 30
readinessProbe: # 就绪探针
httpGet:
path: /health
port: 8000
initialDelaySeconds: 30
periodSeconds: 10
volumes:
- name: model-volume
persistentVolumeClaim:
claimName: model-pvc
nodeSelector: # 调度到GPU节点
accelerator: nvidia-a100
---
```

HPA自动伸缩

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
name: llm-hpa
spec:
scaleTargetRef:
apiVersion: apps/v1
```



```

kind: Deployment
name: llm-inference
minReplicas: 2
maxReplicas: 8
metrics:
- type: Resource
  resource:
    name: gpu-utilization
    target:
      type: AverageValue
      averageValue: "70" # GPU利用率超70%时扩容

```

6.6 生产级监控体系

Prometheus + Grafana 监控方案

1. 在FastAPI中集成Prometheus指标

```

from prometheusclient import Counter, Histogram,
from fastapi.responses import PlainTextResponse

```

定义指标

```

REQUESTCOUNT = Counter(
    'inferencerequests total',
    'Total inference requests',
    ['model', 'status'] # 标签：模型名、状态（成功 / 失败）
)

```

```

REQUESTLATENCY = Histogram(
    'inference latency seconds',
    'Inference latency in seconds',
    ['model'],

```

```

    buckets=[0.1, 0.25, 0.5, 1.0, 2.0, 5.0, 10.0] # 延迟
)

```

```

GPUUTILIZATION = Gauge(

```

```

'gpuutilizationpercent',
'GPU utilization percentage',
['gpuid']
)

```

```

GPUMEMORYUSED = Gauge(
'gpumemoryusedbytes',
'GPU memory used in bytes',
['gpuid']
)

```

```

ACTIVEREQUESTS = Gauge(
'inferenceactiverrequests',
'Number of active inference requests',
['model']
)

```

暴露metrics端点

```

@app.get("/metrics", responseclass=PlainTextResponse)
async def metrics():
    return generate_latest()

```

2. 在推理接口中记录指标

```

@app.post("/predict")
async def predict(request: PredictRequest):
    ACTIVEREQUESTS.labels(model="resnet50").inc()
    start = time.perfcounter()

    try:
        result = await do_inference(request)
        REQUESTCOUNT.labels(model="resnet50", status="success").inc()
        return result
    except Exception as e:
        REQUESTCOUNT.labels(model="resnet50", status="error").inc()
        raise

```

```
finally:
    latency = time.perfcounter() - start
REQUESTLATENCY.labels(model="resnet50").observe(latency)
ACTIVEREQUESTS.labels(model="resnet50").dec()
```

Prometheus 配置

```
prometheus.yml
```

```
global:
```

```
scrapeinterval: 15s
```

```
scrapeconfigs:
```

```
- jobname: 'llm-inference'
```

```
kubernetessdconfigs:
```

```
- role: pod
```

```
relabelconfigs:
```

```
- sourcelabels: [metakubernetespodlabelapp]
```

```
regex: llm-inference
```

```
action: keep
```

关键监控大盘指标：

类别 指标 告警阈值

延迟 P50 / P95 / P99 延迟 P99 > 2s

吞吐 QPS （每秒请求数） < 10 QPS

错误率 5 x x 错误率 > 1%

GPU 利用率 < 30%（浪费）或 > 90%（瓶颈）

显存 使用率 > 85%

队列 等待请求数 > 100

模型 推理耗时分布 趋势上升

6.7 日志与可观测性

```
import structlog
```

```
import json
```

```
from datetime import datetime
```

结构化日志

```
logger = structlog.get_logger()

@app.post("/predict")
async def predict(request: PredictRequest):
    request_id = generaterequestid()

    log = logger.bind(
        request_id=request_id,
        model="resnet50",
        inputs_size=len(request.inputdata),
    )

    log.info("inference start")
    start = time.perfcounter()

    try:
        result = await doinference(request)
        latency_ms = (time.perfcounter() - start) * 1000

        log.info("inferencesuccess",
            latency_ms=round(latency_ms, 2),
            confidence=result.confidence)
        return result
    except Exception as e:
        log.error("inference failed", error=str(e))
        raise
```

日志输出格式（JSON，方便ELK/Loki收集）：

```
{
  "timestamp": "2025-01-15T10:30:00Z",
  "level": "info",
  "event": "inferencesuccess",
  "request_id": "req-abc123",
  "model": "resnet50",
```

```
"latencymms": 45.23,
"confidence": 0.95
}
```

- - -

七、模型评测体系

7.1 LLM 评测基准

评测维度 基准 说明 评测方式

通用知识 MMLU 57个学科的多选题 4选1准确率

中文能力 C-Eval 中文多学科评测 4选1准确率

代码能力 HumanEval、MBPP 代码生成正确率 pass@k

数学推理 GSM8K、MATH 数学题求解 准确率

长文本 LongBench 长文本理解与生成 多任务综合

指令遵循 IFEval 是否遵循指令格式 规则匹配

安全性 TruthfulQA 是否给出真实答案 真实率

人类偏好 Chatbot Arena 人类盲评排行榜 ELO评分

工具调用 ToolBench, BFCL 准确调用API能力 执行成功率

RAG能力 RAGAS 检索增强生成质量 Faithfulness, Relevancy

7.2 使用 lm-evaluation-harness 进行评测

安装

```
pip install lm-eval
```

评测MMLU

```
lmeval --model hf \
--modelargs pretrained=Qwen/Qwen2.5-7B \
--tasks mmlu \
--batchsize 8
```

评测GSM8K（数学推理）

```
lmeval --model hf \
--modelargs pretrained=Qwen/Qwen2.5-7B \
```

```
-- tasks gsm8k \
-- batchsize 4
```

评测HumanEval（代码生成）

```
lmeval --model hf \
--modelargs pretrained=Qwen/Qwen2.5-7B,trustremote_code \
--tasks humaneval \
--batchsize 4
```

多任务评测

```
lmeval --model hf \
--modelargs pretrained=Qwen/Qwen2.5-7B \
--tasks mmlu,gsm8k,humaneval \
--outputpath ./evalresults
```

7.3 RAGAS——RAG系统专用评测

```
from ragas import evaluate
from ragas.metrics import (
    faithfulness, # 忠实度：回答是否基于检索内容
    answer_relevancy, # 答案相关性：回答是否切题
    context_recall, # 上下文召回：是否检索到所有相关信息
    context_precision, # 上下文精度：检索内容是否都是相关的
)
```

评测数据格式

```
evaldata = {
    "question": [ "什么是机器学习？" , "深度学习和机器学习的区别？" ],
    "answer": [ "机器学习是AI的子集..." , "深度学习使用神经网络..." ],
    "contexts": [ [ "机器学习是让计算机从数据中学习的技术..." ],
                  [ "机器学习是传统方法，深度学习是其子集..." ] ],
    "groundtruth": [ "机器学习是让计算机从数据中自动学习规律的技术" ,
                     "深度学习是机器学习的子集，使用多层神经网络" ]
}
```

运行评测

```
result = evaluate(  
    dataset=evaldata,  
    metrics=[faithfulness, answerrelevancy, contextreca  
    llm=ChatOpenAI(model="gpt-4"), # 使用LLM作为评判  
)
```

```
print(result)
```

输出: {'faithfulness': 0.85, 'answerrelevancy': 0.9
'contextrecall': 0.78, 'contextprecision': 0.88}

7.4 推理性能评测

```
import time
```

```
import numpy as np
```

```
def benchmark_inference(model, input_shape, nwarmup
```

```
"""推理性能基准测试"""
```

```
dummy_input = torch.randn(input_shape).to(device)
```

```
# 预热 (GPU需要预热)
```

```
for i in range(nwarmup):
```

```
    with torch.no_grad():
```

```
        _ = model(dummy_input)
```

```
# 正式测试
```

```
latencies = []
```

```
for i in range(niterations):
```

```
    torch.cuda.synchronize()
```

```
    start = time.perfcounter()
```

```
    with torch.no_grad():
```

```
        _ = model(dummy_input)
```

```
    torch.cuda.synchronize()
```

```
    latency = (time.perfcounter() - start) * 1000 # ms
```

```
latencies.append(latency)

latencies = np.array(latencies)
print(f"平均延迟: {latencies.mean():.2f}ms")
print(f"P50延迟: {np.percentile(latencies, 50):.2f}")
print(f"P99延迟: {np.percentile(latencies, 99):.2f}")
print(f"吞吐量: {1000/latencies.mean():.1f} QPS")

- - -
```

八、模型版本管理与 A / B 测试

8.1 模型注册中心

模型注册中心 (Model Registry) 的核心功能：

1. 版本管理：每个模型有多个版本，记录训练配置、数据集、指标
2. 阶段管理：None → Staging → Production → Archived
3. 元数据：训练参数、评估指标、模型签名
4. 访问控制：谁可以发布 / 部署模型

使用 MLflow 模型注册中心

```
import mlflow
```

记录模型到注册中心

```
with mlflow.start_run():
    mlflow.log_params({
        "modeltype": "resnet50",
        "learningrate": 1e-3,
        "epochs": 20,
        "dataset": "cifar10",
    })
    mlflow.log_metrics({
        "valaccuracy": 0.953,
        "valloss": 0.142,
        "inference latencyms": 12.5,
    })
```



```
mlflow.pytorch.logmodel(
    model,
    "model",
    registered_model_name="imageclassifier_v2"
)
```

模型阶段管理

```
client = mlflow.tracking.MlflowClient()
```

将版本1移到Staging

```
client.transition_model_version_stage(
    name="imageclassifier_v2",
    version=1,
    stage="Staging"
)
```

Staging测试通过后，移到Production

```
client.transition_model_version_stage(
    name="imageclassifier_v2",
    version=1,
    stage="Production"
)
```

获取当前生产版本的模型

```
model_uri = "models:/imageclassifier_v2/Production"
model = mlflow.pytorch.load_model(model_uri)
```

8.2 A / B 测试与灰度发布

A / B测试流程：

```

用户请求 → 路由层 → ┬── 模型 A（当前版本） 90%流量
                    └── 模型 B（新版本） 10%流量
↓
收集两组的延迟、准确率、用户反馈
↓

```

统计检验：B是否显著优于A？

↓

是 → 逐步增加B的流量到100%

否 → 回滚到A

A / B测试路由实现

```
import hashlib
from fastapi import Request

class ModelRouter:
    """ 模型路由器 - 支持A / B测试和灰度发布 """

    def __init__(self):
        self.models = {
            "modelv1": {"weight": 90, "model": loadmodel("v1")},
            "modelv2": {"weight": 10, "model": loadmodel("v2")},
        }

    def route(self, request: Request) -> str:
        """ 根据流量比例路由到不同模型版本 """
        # 使用用户ID或请求ID做一致性哈希
        userid = request.headers.get("X-User-ID", str(id(request)))
        hashval = int(hashlib.md5(userid.encode()).hexdigest(), 16)

        cumulative = 0
        for modelname, config in self.models.items():
            cumulative += config["weight"]
            if hashval < cumulative:
                return modelname

        return "modelv1" # 默认
```

FastAPI集成

```
@app.post("/predict")
async def predict(request: PredictRequest, httprequest: Request):
    modelname = router.route(httprequest)
```

```
model = router.models[modelname]["model"]

result = await model.predict(request)
result["modelversion"] = modelname # 记录使用的模型版本
return result
```

8.3 CI/CD 流水线

```
.github/workflows/model-deploy.yml
name: Model Deploy Pipeline

on:
  push:
  paths:
    - 'models/'

jobs:
  evaluate:
    runs-on: gpu-runner
    steps:
      - uses: actions/checkout@v4

      - name: Run Model Evaluation
        run:
          python evaluate.py \
            --model-path $MODEL_PATH \
            --benchmark mmlu,gsm8k \
            --output evalresults.json

      - name: Check Quality Gate
        run:
          # 质量门槛：新模型必须不低于当前生产模型
          python checkqualitygate.py \
            --new-results evalresults.json \
            --baseline prodresults.json \
            --min-improvement 0.01
```

```
export:
needs: evaluate
runs-on: gpu-runner
steps:
- name: Export to ONNX
run:
python exportonnx.py --model $MODEL_PATH --opset 1

- name: Optimize with TensorRT
run:
trtexec --onnx=model.onnx --fp16 --saveEngine=model

- name: Run Performance Benchmark
run:
python benchmark.py --engine model.trt --max-late

deploy-staging:
needs: export
runs-on: ubuntu-latest
steps:
- name: Deploy to Staging
run:
kubectl apply -f k8s/staging/
kubectl rollout status deployment/llm-staging

- name: Run Smoke Tests
run:
python smoketest.py --endpoint http://staging:8000

deploy-production:
needs: deploy-staging
runs-on: ubuntu-latest
steps:
- name: Canary Deployment (10% traffic)
```

```

run:
kubectl apply -f k8s/canary/
sleep 300 # 观察5分钟

- name: Check Metrics
run:
python checkcanarymetrics.py \
--error-rate-threshold 0.01 \
--latency-threshold-ms 100

- name: Full Rollout
if: success()
run:
kubectl apply -f k8s/production/

- name: Rollback on Failure
if: failure()
run:
kubectl rollout undo deployment/llm-production

- - -

```

九、端到端部署流水线

训练模型 (PyTorch)

↓ 导出

ONNX 中间格式

↓ 验证+优化

↓ 转换

├── TensorRT Engine (GPU部署)

├── OpenVINO IR (Intel CPU部署)

└── ONNX Runtime (通用部署)

↓ 封装

推理服务 (FastAPI / Triton / vLLM)

↓ 容器化

Docker / K8s 部署

↓ 监控

性能监控 + A / B 测试 + 模型版本管理

9.1 完整部署检查清单

部署前检查：

- ☐ 模型精度验证：在测试集上评估，确保精度达标
- ☐ ONNX 导出验证：数值误差 $< 1e-5$
- ☐ 量化精度验证：量化后精度损失 $< 1\%$
- ☐ 推理延迟测试：满足 SLA 要求
- ☐ 内存 / 显存测试：不超过资源限制
- ☐ 压力测试：在目标 QPS 下稳定运行 30 分钟
- ☐ 安全性检查：输入验证、输出过滤
- ☐ 日志规范：关键操作有结构化日志
- ☐ 监控告警：延迟、错误率、资源使用告警配置
- ☐ 回滚方案：一键回滚到上一个稳定版本
- ☐ 文档更新：API 文档、模型卡片更新

9.2 成本优化策略

推理成本 = GPU 单价 × GPU 数量 × 使用时间

优化方向：

1. 量化：FP16 → INT8 → INT4

显存减少 50 - 87%，吞吐提升 2 - 4 倍

成本节省：40 - 70%

2. 动态批处理

将多个请求合并为一个 batch

吞吐提升 2 - 5 倍（取决于请求频率）

成本节省：30 - 50%

3. 弹性伸缩

低峰期减少副本，高峰期自动扩容

K8s HPA + 自定义指标

成本节省：30 - 60%（取决于流量波动）

4. Spot / Preemptible实例

使用竞价实例（价格低60 - 90%）

需要处理实例被回收的情况

成本节省：60 - 90%

5. 模型缓存

缓存常见请求的结果

对FAQ类场景特别有效

成本节省：20 - 50%

综合优化案例：

7B模型，1000 QPS

优化前：8张A100，月费约\$20,000

优化后：2张A100 + AWQ量化 + 动态批处理 + 弹性伸缩

月费约\$5,000

成本降低75%

- - -

十、硬件适配建议

硬件 推荐方案 典型配置

NVIDIA A100 / H100 TensorRT + FP16 / BF16 企业级推理

NVIDIA RTX 4090 vLLM + GPTQ 4bit 个人 / 小团队

Intel Xeon CPU OpenVINO + INT8 CPU推理

边缘设备 ONNX Runtime + INT8量化 IoT场景

手机端 TFLite / NCNN / MNN 移动端APP

- - -

十、硬件适配建议

硬件 推荐方案 典型配置 月成本估算

NVIDIA A100/H100 TensorRT + FP16/BF16 企业级推理 \$3,000
NVIDIA RTX 4090 vLLM + AWQ 4bit 个人/小团队 \$500 - 1,500
Intel Xeon CPU OpenVINO + INT8 CPU推理 \$200 - 800
边缘设备 (Jetson) TensorRT + INT8 IoT场景 \$100 - 300
手机端 TFLite / NCNN / MNN 移动端APP 设备成本

10.1 不同规模业务的部署架构

小型团队 (<100 QPS) :

Ollama / vLLM 单卡 → FastAPI → Docker

成本: 1张RTX 4090 (~\$2000一次性 或 ~\$500/月云服务器)

中型业务 (100 - 1000 QPS) :

vLLM 多卡 → K8s编排 → Prometheus监控 → Nginx负载均衡

成本: 2 - 4张A100 (~\$3000 - 6000/月)

大型业务 (>1000 QPS) :

Triton推理集群 → Istio服务网格 → 分布式追踪 → 自动伸缩

成本: 8 - 32张A100/H100 (~\$20,000 - 80,000/月)

- - -

十一、AI 应用安全

11.1 推理服务安全防护

输入验证与清洗

```
class InputValidator:
```

```
""" 推理服务输入验证器 """
```

```
MAXINPUTLENGTH = 10000 # 最大输入长度
```

```
MAXBATCHSIZE = 32 # 最大批量大小
```

```
@classmethod
```

```
def validate_text_input(cls, text: str) -> tuple:
```

```
""" 验证文本输入 """
```



```

if not text or not text.strip():
    return False, "输入不能为空"
if len(text) > cls.MAXINPUTLENGTH:
    return False, f"输入长度超过限制({cls.MAXINPUTLENGTH})"

# 检测注入攻击
injectionpatterns = [
    "ignore previous instructions",
    "system prompt",
    "you are now",
    "forget your role",
]
lowertext = text.lower()
for pattern in injectionpatterns:
    if pattern in lowertext:
        return False, f"检测到潜在注入攻击"

return True, "OK"

```

输出过滤

```

class OutputFilter:
    """推理结果过滤"""

    SENSITIVEPATTERNS = [
        r'\b\d{16}\b', # 信用卡号
        r'\b\d{17}[\dXx]\b', # 身份证号
        r'\b[w+@\w+\.\w+]\b', # 邮箱地址
    ]

    @classmethod
    def filtersensitiveinfo(cls, text: str) -> str:
        """过滤敏感信息"""
        import re
        for pattern in cls.SENSITIVEPATTERNS:
            text = re.sub(pattern, '[REDACTED]', text)

```

```
return text
```

速率限制

```
from slowapi import Limiter
limiter = Limiter(keyfunc=getremoteaddress)

@app.post("/predict")
@limiter.limit("100/minute") # 每分钟最多100次请求
async def predict(request: Request, predictrequest: Request):
    # 输入验证
    valid, msg = InputValidator.validate_text_input(predictrequest)
    if not valid:
        raise HTTPException(status_code=400, detail=msg)

    result = await do_inference(predictrequest)

    # 输出过滤
    result.content = OutputFilter.filtersensitive_info(result.content)
    return result
```

11.2 模型安全风险

风险 说明 防御措施

对抗攻击 添加微小扰动使模型误判 对抗训练、输入预处理

数据投毒 恶意数据污染训练集 数据清洗、来源验证

模型窃取 通过API查询反推模型 速率限制、输出扰动

Prompt注入 恶意输入劫持模型行为 输入验证、指令隔离

隐私泄露 模型记忆并泄露训练数据 差分隐私、数据脱敏

- - -

十二、未来趋势

1. 统一部署框架：Apache TVM 等工具统一不同后端

2. Speculative Decoding：小模型猜测+大模型验证，加速LLM推理2-3倍

3. 异构计算：CPU + GPU + NPU 协同推理

- 4 . K V C a c h e 优化：P a g e d A t t e n t i o n、前缀缓存、K V C a c h e 压缩
- 5 . 模型压缩联合优化：剪枝 + 蒸馏 + 量化同时进行
- 6 . S e r v e r l e s s A I：按需弹性伸缩的 A I 推理服务
- 7 . 边缘智能：端侧大模型推理（手机 / P C 运行 7 B 模型）
- 8 . 多模态推理优化：图文音视频统一部署框架
- 9 . M o E 部署：稀疏激活模型的高效推理服务

- - -

核心总结：模型部署的核心链路是 " 训练→导出→优化→服务→监控→迭代 "。O N N X 实现跨框架互通，T e n s o r T r a c k 实现 GPU 极致性能，v L L M 是 L L M 高吞吐推理的标配。量化（F P 1 6 / I N T 8 / I N T 4）是提升推理效率的关键，蒸馏进一步压缩模型。生产部署需要关注：容器化（D o c k e r / K 8 s）、监控（P r o m e t h e u s / G r a f a n a）、A / B 测试、安全防护。选择部署方案需根据硬件环境、性能需求、成本预算和团队能力综合考量。记住：精度只是起点，工程化落地才是终点 *。